

QCon

全球软件开发大会

让推理引擎可被“看见”

大模型推理端到端Token级可观测工程实践

刘杨

蚂蚁集团可观测架构师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 推理全链路可观测体系

02 传统Trace的局限

03 Token为中心的可观测方案

04 经典案例

05 社区贡献与展望

06 QA

01

推理全链路可观测体系

以推理为核心的统一观测解决方案



推理云服务

推理云核心组件全覆盖

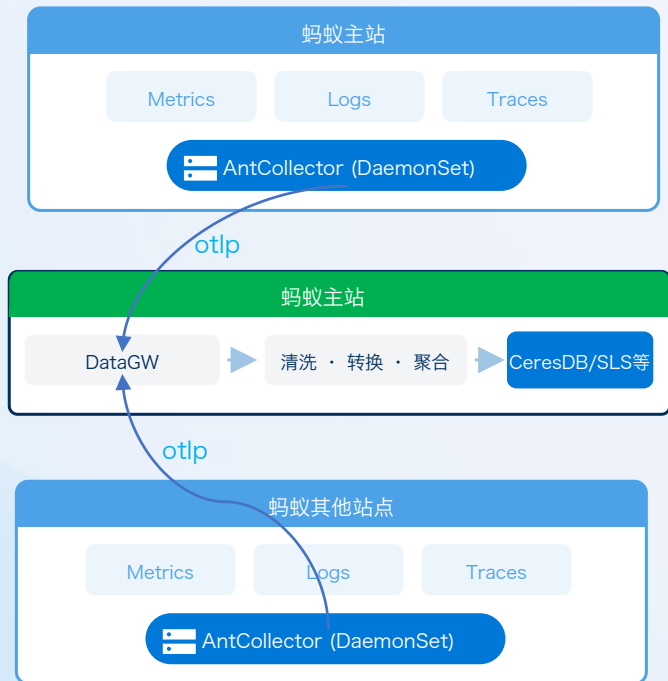


AI应用

多语言跨协议端到端链路追踪

推理云服务：混合云架构 · 核心组件覆盖

推理混合云架构



推理核心组件监控覆盖



推理云服务：标准化-统一口径 · 消除孤岛



统计/链路指标标准化 — 核心性能指标



TTFT

Time to First Token

首 Token 延迟

用户等待体验核心指标

覆盖排队 + Prefill 计算

P99



TPOT

Time Per Output Token

单 Token 生成时间

流式输出流畅度指标

Decode + KV Cache 效率

AVG



E2E

End-to-End Latency

端到端时延

全链路服务质量指标

网络 + 排队 + 推理全覆盖

SLA



严格成功率 — 超越 HTTP 200

非严格成功的异常类型：

- RequestError 请求失败
- EmptyOutput 模型输出为空
- ErrorCode>80% 乱码超阈值
- Repeat>80% 重复超阈值
- TTFT/TPOT 延迟超阈值



规范统一收益 — 打破数据孤岛

统计&链路指标规范



精准 FO 支

撑标准化指标为引擎实例/AIGW 节点故障自愈提供决策依据



弹性扩缩容

基于排队数/TPOT 等弹性策略，实现资源按需使用



统一监控大盘

SRE 与业务方共享同一视角，全平台横向对比

AI应用：端到端分布式追踪

端到端推理链路



request_id 精准关联

解决问题

单 Traceld 关联多笔推理请求时的精准定位

- AIGW 生成全局唯一 ID
- 下游组件日志 + Span 统一透传
- 监控指标 ↔ 链路详情一键跳转

打通指标 → 明细 → Trace 完整闭环

异构协议串联

解决问题

SOFATracer 与 OpenTelemetry 两种协议 Span 串联

- SOFA-Traceld → Parent-Traceld
- SOFA-RpcId → Parent-RpcId 映射
- 查询时自动扫描跨协议关联

双向可追溯 · 语义一致性保障

多语言接入

接入策略

日志转 Span + 原生 OTLP 并行，按场景选择

Java SOFATracer 日志 → AntCollector → Span

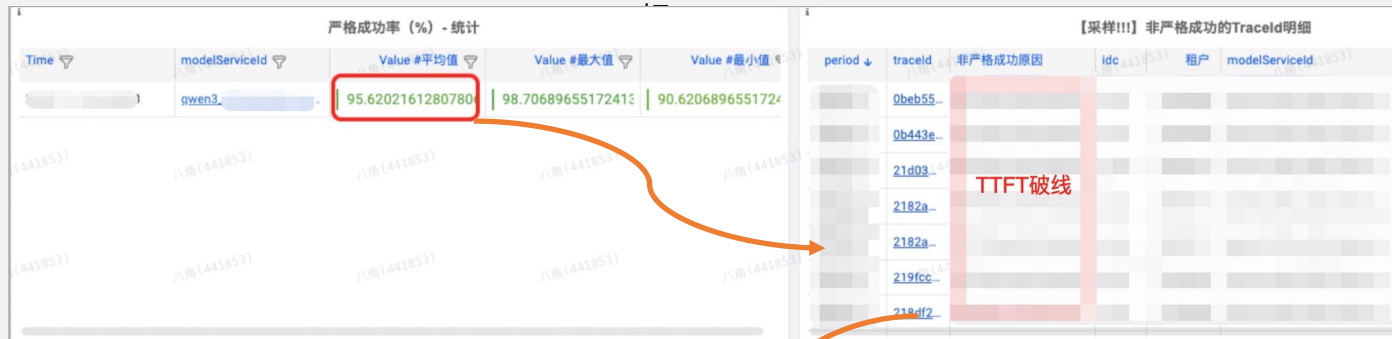
Go 原生 OTLP

Python 原生 OTLP

渐进式 OTel 覆盖 · 最终全链路统一

产品效果：指标下探Trace，快速定界

AIGW视角推理服务统计指



引擎黑洞?

单Trace链路



02

传统Trace的局限

Token生产不可见

大模型推理的核心体验指标



TTFT

Time To First Token

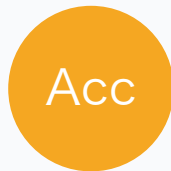
首字响应时间
用户等多久看到第一个字输出



TPOT

Time Per Output Token

每Token平均输出时间
后续字输出快不快，有无卡顿



精度指标

Accuracy Metrics

回答好不好
答非所问/乱码/复读等异常

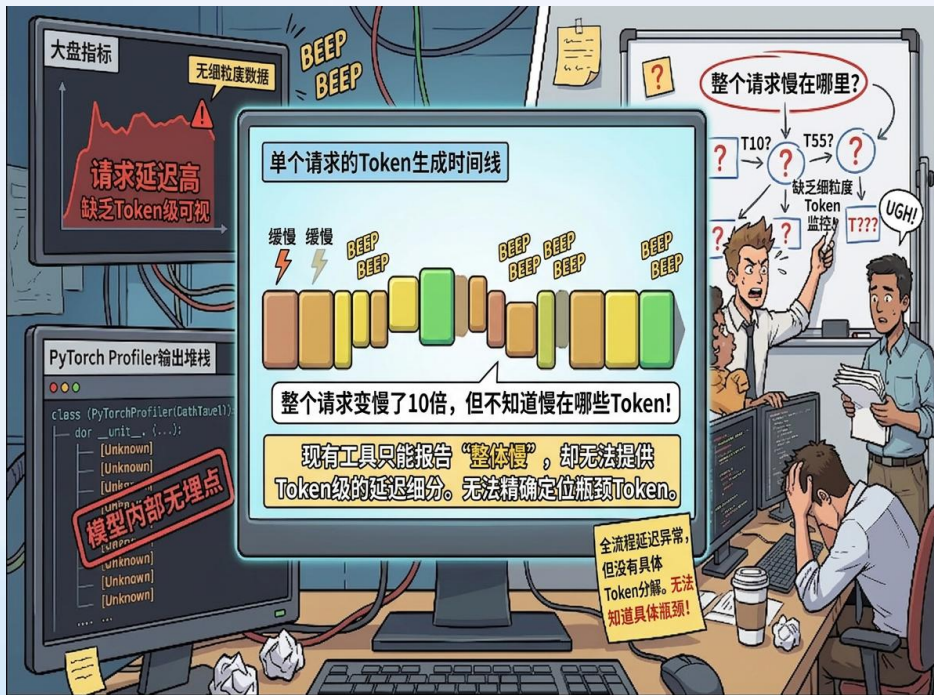


核心论点

Key Insight

衡量大模型服务
性能看Token速度，质量看
Token分布与选择

性能异常：请求变慢，慢在哪



性能问题的难点

引擎黑盒

传统Trace到请求粒度截止，而回复是由Token构成，逐次吐出来

并发干扰可见

不知道本请求被谁阻塞、怎么阻塞，大请求对小请求的拖累完全不透明

关键拆解：慢在哪些Token → 其他请求的干扰

精度难题：答非所问、乱码、复读



为什么精度问题难排查？

随机性导致不可复现

同一Prompt，两次调用可能结果迥异，无法稳定复现

错误从某Token蔓延

一旦某Token出错，后续Token受其影响，像多米诺效应

采样参数是黑手

Temperature过高→乱码，过低→复读；错误参数隐匿其中

BOS等特殊Token不显示

特殊Token在网关/引擎日志里面不显示

关键需求：每个Token的概率分布是怎样的

归根结底是 Token生产的问题

性能异常

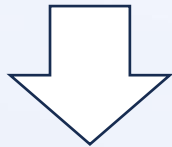
某些Token生产耗时过长

并发退化

Token粒度上相互争抢

精度异常

某个Token被错误选中



以Token为中心的引擎可观测

我们的愿景

打造一款"显微镜"

下探到Token粒度，透视引擎内部的每一个细节

让延迟与精度问题无处遁形

03

Token为中心可观测方案

核心技术挑战

1

恰当的数据采集

- 深入理解引擎调度逻辑，结合业务场景精选关键数据
- 粒度太粗无法定位Token级问题；太细则存储与性能开销爆炸
- 需精准平衡数据粒度与采集成本，有效刻画影响性能的关键因素

2

复杂异构环境适配

- 三大引擎/三种AI硬件/多种部署架构/复杂并行与多样调度策略
- 必须深入理解AI Infra各层细节，逐一适配采集逻辑
- 最终输出统一标准的可观测数据

3

低性能开销要求

- 硬件昂贵，不能影响吞吐，也不能影响TTFT/TPOT等体验指标
- 生产问题难以线下复现，必须实时在线大规模采集
- 为还原请求间相互干扰，不能做采样，需全量采集

核心矛盾：既要深入引擎内部获取Token级精细数据，又要在复杂异构环境中保持极致低开销的实时在线全量采集

恰当的数据采集：推理引擎迭代循环

1 选取请求，组成批次
从等待队列中挑选请求，组成本次迭代的batch

2 批量执行处理
GPU上执行一次forward

3 每个请求生成一个Token
Sampler从logits中采样，输出一个Token

4 进入下一代，循环往复
回到步骤1，直到所有请求完成生成

❗ 核心挑战
每次迭代的batch组成不同，请求间干扰模式不同，定位极难



恰当的数据采集：在粒度与开销之间精准平衡

请求级 (Request) ← 传统Trace已支持

Token级 (Token) ← 我们采集到这里

阶段级 (Phase: 预处理/排队/KVCache/模型forward) ← 我们采集到这里

层级 (Layer: Attention / MLP / MOE) (不采集)

Kernel级 (CUDA Kernel) (不采集)

太粗（仅请求级）

无法回答Token级问题，精度问题/并发干扰无从排查

太细（到Kernel级）

存储开销爆炸，在线采集不现实，影响推理性能

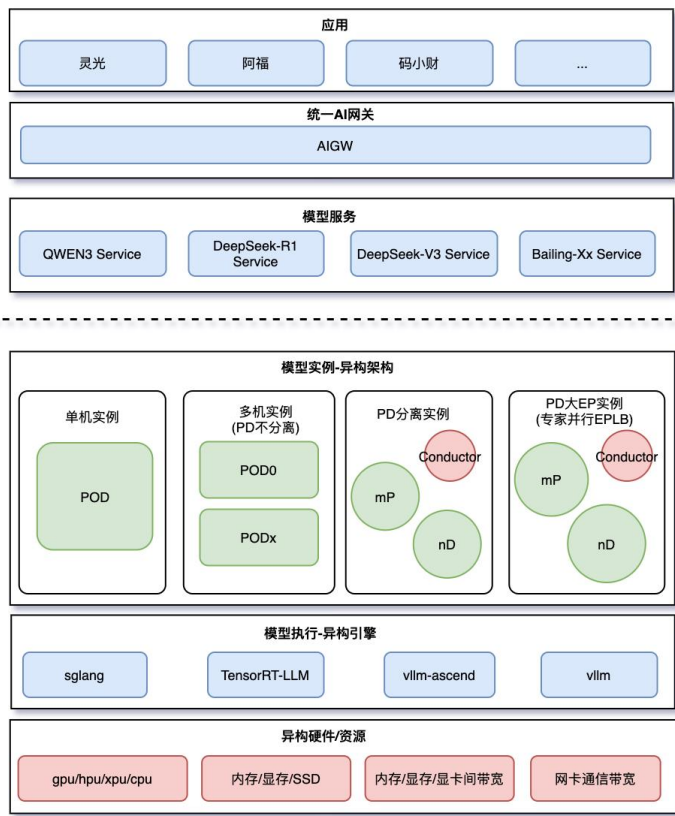
我们的选择

仅采集Token粒度及其子阶段数据

恰当的数据采集：Token粒度性能关键数据

分类	指标	口径	解决什么问题
Token 维度	进迭代时刻	请求被调度到引擎迭代的时间	定位调度延迟
	出迭代时刻	引擎完成迭代计算，吐出Token的时间	定位计算延迟
	Token执行耗时	出迭代时刻 - 进迭代时刻	真正在迭代里的执行耗时
	Token调度时延	当前Token进迭代 - 上个Token出迭代	捕获Decode被Prefill中断等场景
	Token总耗时	当前Token出迭代 - 上个Token出迭代	用户实际感知的Token产出耗时
执行批 维度	并发请求数	迭代执行批的请求数	粗略刻画迭代负载
	并发Token总数	迭代执行批Token数目	精确刻画算力/显存带宽需求

异构挑战：多硬件 × 多引擎 × 多部署架构



应用层

灵光 · 阿福 · 码小财



网关与模型服务

AIGW -> ModelService



模型实例 — 异构架构

单机实例 → 多机实例(PD不分离) → PD分离实例 → PD大EP实例

Conductor调度 · 专家并行EPLB · mP/nD分离



模型执行 — 异构引擎

sglang · TensorRT-LLM · vllm-ascend · vllm



异构硬件资源

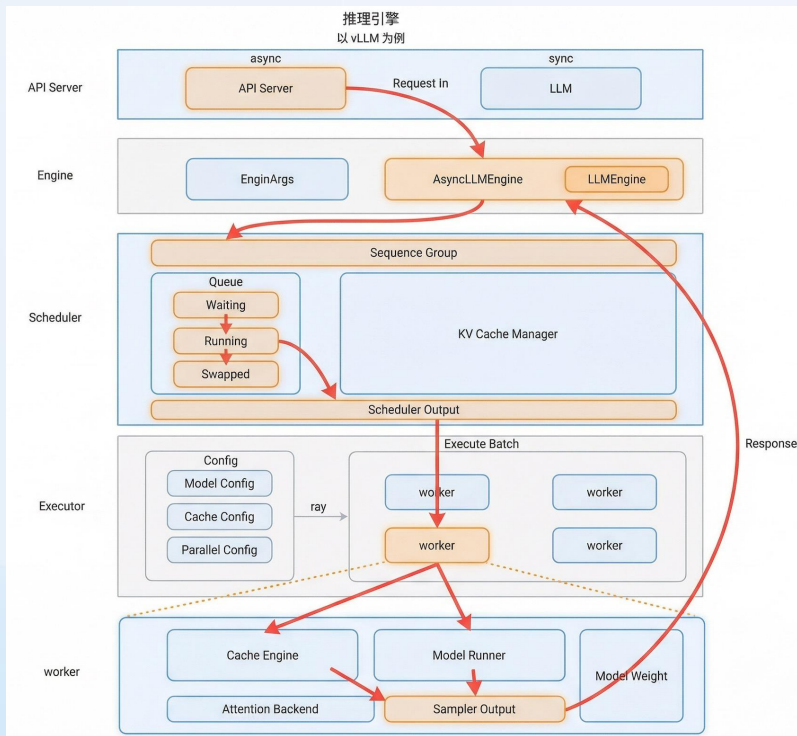
GPU/HPU/XPU/CPU · 内存/显存/SSD · 显卡间带宽 · 网卡通信带宽



可观测性必须跨引擎、跨硬件、跨部署模式统一



低性能开销：技术方案选择



无侵入探针方案（未选）

无法获取引擎内部Token粒度上下文

引擎迭代快，Hook函数签名可能变化，维护成本较高

性能不占优势，友商实践百分点开销

侵入式直接埋点（我们的选择）

直接获取引擎内部丰富上下文

Token粒度精确数据

同步维护引擎代码，稳定可靠

黄色部分为埋点组件，核心执行层基本无侵入，埋点主要集中在API/调度层，最大限度减少性能影响

低性能开销：极致优化-千分点损失



挑战：768高并发压测下，开启Token粒度可观测后TTFT增长10%+

对于推理服务，10%的TTFT增长意味着用户体验的明显下降 — 这是不可接受的

四大优化策略

1 埋点逻辑上移

大部分处理放到上层API/调度层，而非引擎核心执行层
避免在热路径上增加计算开销

2 单请求单Trace节点上报

Token粒度数据作为属性附着到请求Trace节点
避免大量Trace节点上报开销，产品侧后置还原

3 高性能序列化 + 批量操作 + 复用

json → orjson 替换，数组批量追加避免反复生成新数组
复用引擎已转换好的detokenize数据

4 异步批量上报 + 完成时上报

Trace上报批量异步执行，不影响主流程
只在请求完成时上报，利用推理服务低QPS特点

TTFT/TPOT千分点损失，吞吐等成本指标无损

生产级大规模运行

推全可观测Trace功能，覆盖vllm-ascend / sglang / TRT公共服务
大规模实时在线采集，无因可观测功能导致的任何故障及性能问题

产品方案：引擎显微镜-广角镜 VS 高倍镜

从“混沌与盲猜”到“清晰与精准”



引擎并发剖析（广角镜头）

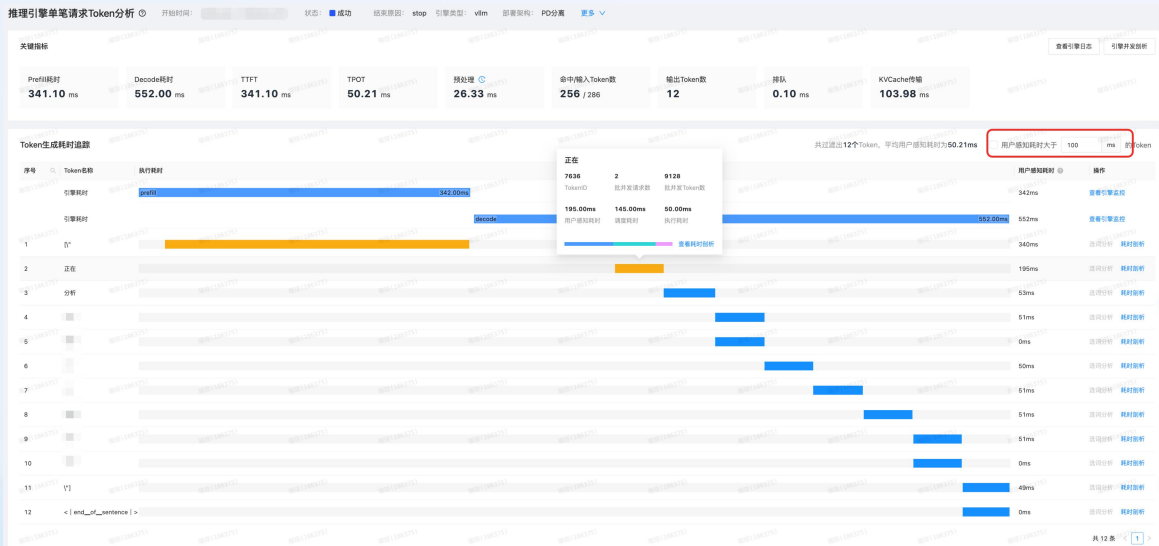
多轨道瀑布图，清晰呈现引擎所有请求的并发、竞争和协作关系，快速识别资源冲突与性能瓶颈。

引擎Token分析（高倍镜）

切换到单个请求角度，观察单Token生成耗时，可视化候选词概率分布，精准定位延迟与精度根源。

业界首个覆盖多引擎、Token级的深度可观测Trace产品，领先业界，独此一家

产品方案-Token分析：拆解每个Token耗时



核心功能

核心指标

输入/输出/排队/TTFT/TPOT/KVCache传输/预处理等核心指标

Token执行耗时

每个Token的执行耗时、调度延迟精确到毫秒级

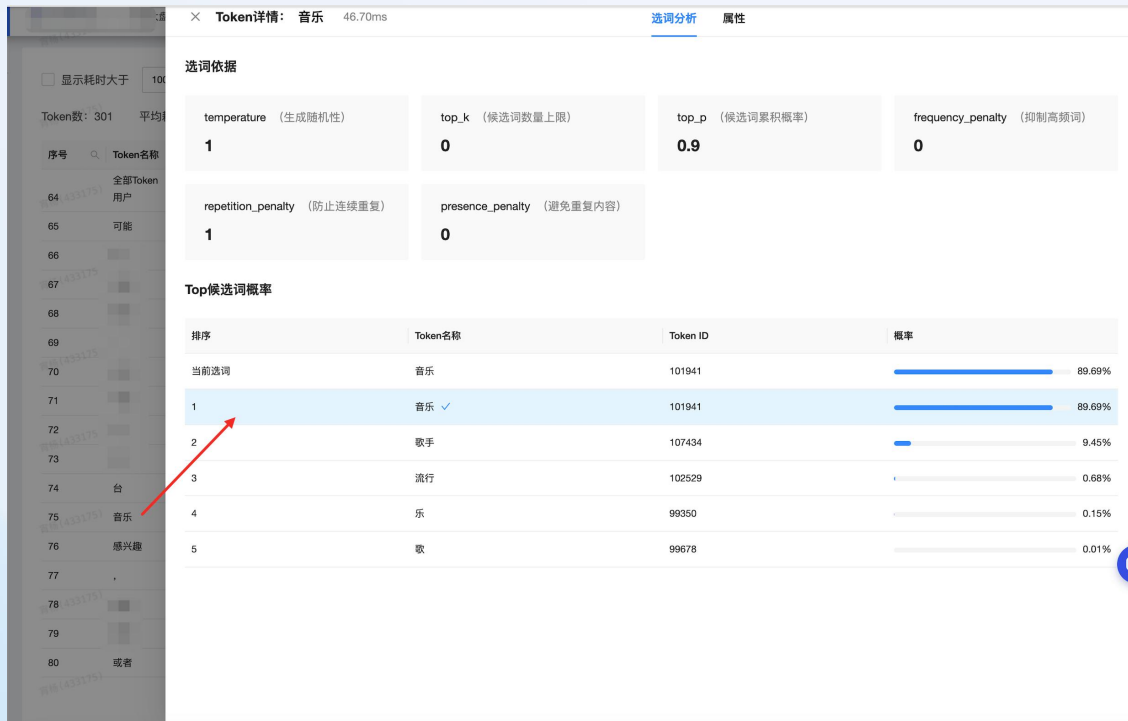
Token执行所在批负载

Token生产所在批的负载情况

关键拆解：拆解到Token，让性能问题定位不盲目



产品方案-选词分析：透视每一个Token背后的模型决策



能看到什么？

当前被选中的Token

及其在候选词中的排名

Top-K候选词概率分布

展示前N个候选词的概率值和Token ID

采样参数展示

Temperature / Top-P / Repetition-Penalty 等

洞察：模型问题还是采样参数的不合理

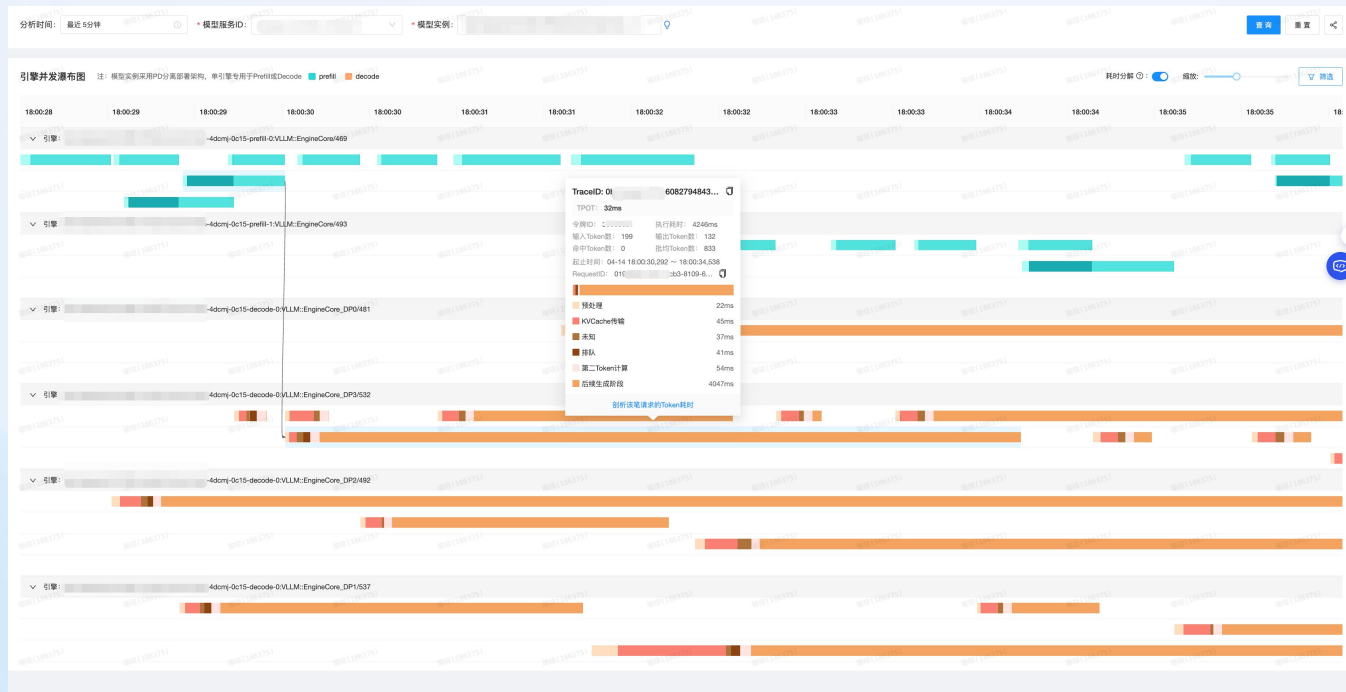
产品方案-并发瀑布图：可视化多请求并发

▸ 多轨道瀑布图

每条轨道代表一台机器，色块展示请求执行全过程

▸ 慢请求条件筛选

可按TTFT/TPOT/输入Token数等条件高亮慢请求



亮蓝色->Prefill

橙色->Decode

04 经典案例

真实线上例子

案例1: TPOT破线

sglang引擎 · 非PD分离

⚠ TPOT 54.77ms 破线

⚠ Token#125 调度耗时 6816ms

👁 用户感知耗时 6.85s

推理引擎单笔请求Token分析

开始时间: 2025-12-09 17:38:27,484

状态: 成功

引擎类型: sglang

部署架构: 非PD分离

更多

关键指标

引擎并发剖析

Prefill耗时

474.00 ms

Decode耗时

7.29 s

TTFT

474.48 ms

TPOT

54.77 ms

预处理

0.42 ms

命中/输入Token数

10293/16261

输出Token数

134

排队

4.18 ms

KVCache传输

0 ms

Token生成耗时追踪

共过滤出2个Token, 平均用户感

序号 Token名称 执行耗时

引擎耗时 474.00ms decode

1

125 replace_all</arg_key>
<arg_value>false</arg_v
alue>

第125个token调度花了6.8秒!

replace_all</arg_key>
<arg_value>false</arg_value>

8155 2 77580

TokenID 批并发请求数 批并发Token数

6.85s 6816.20ms 29.50ms

用户感知耗时 调度耗时 执行耗时

查看耗时剖析

选词分析 耗时剖析

选词分析 耗时剖析

共 2 条 < 1 >

💡 发现异常: 单个Token用户感知耗时高达6.8秒 → 转到并发视角寻找根因

根因：Decode被新请求的Prefill中断

⚠ 请求2的Decode被请求1的Prefill中断

⚡ 非PD分离下Prefill优先级高于Decode

方案：启用PD分离 / 调整调度优先级

引擎并发剖析

[帮助文档](#)

分析时间: 2025-12-09 17:33:27 ~ 2025-12-09 17:43:27

* 模型服务ID:

* 模型实例:

查询

重置

🔍

引擎并发瀑布图

注：非PD分离部署架构，单引擎同时处理Prefill和Decode

prefill decode

耗时分解

缩放

🔍 筛选



并发瀑布图揭示的根因





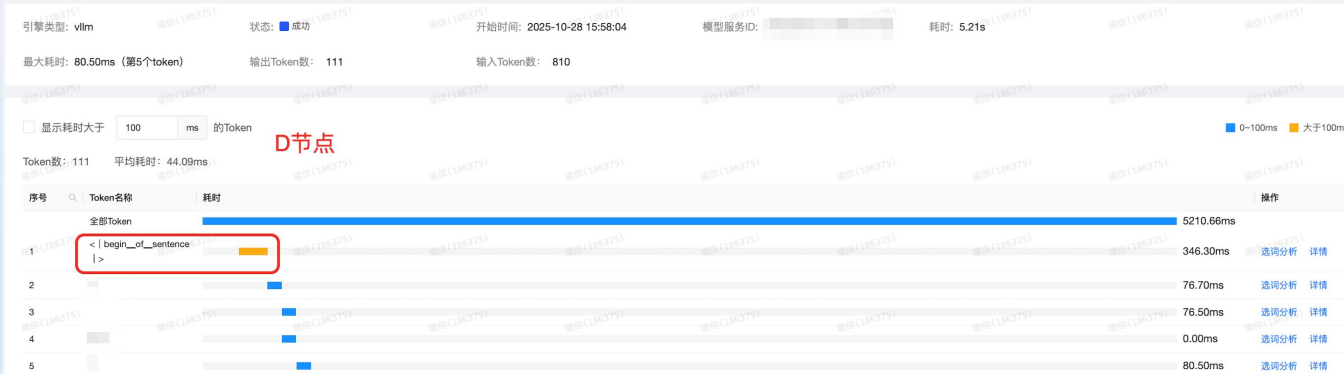
案例2：答非所问

用户投诉：问的是医疗问题，模型回复了leetcode解题答案

推理引擎Token分析



推理引擎Token分析



传统手段难点



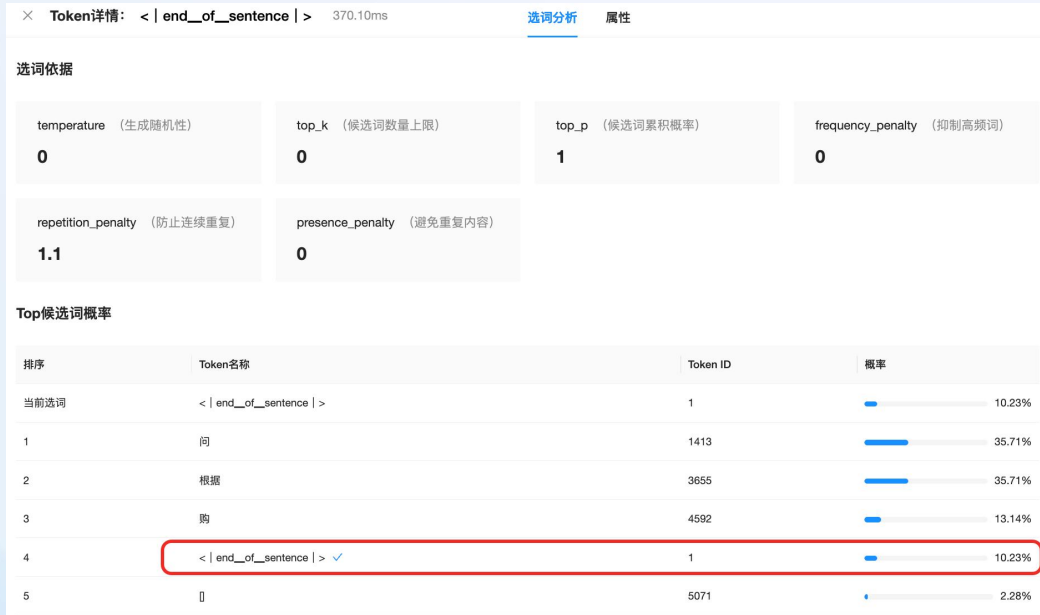
EOS/BOS在日志和网关中显示为""
没有Token分析，根本无法发现这个关键信号

定界

- ➔ P节点，首Token是end_of_sentence
- ➔ D节点，吐出Token是begin_of_sentence
- ➔ BOS出现 = 后续回答与Prompt完全无关



深入：P节点为什么吐出EOS



现象

选中了第四位Token，不是前三

模型问题

EOS概率高达10%，后续也确认是模型badcase

不低的发生概率

大概10%的发生概率



缓解手段：控制采样参数，尽量选择概率靠前的候选Token。



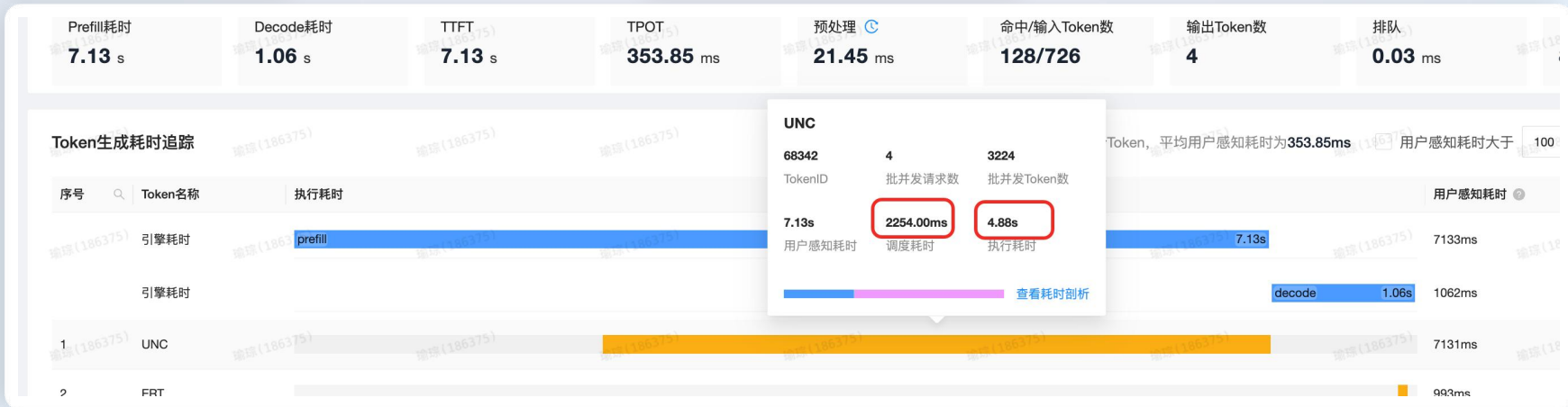
案例3: 小请求TTFT破线

输入Token 700+

🕒 TTFT 7s+

⚠️ 首Token调度等待 2.25s

⚡ 调度后执行耗时 4.88s (正常应<500ms)



💡 发现异常: 最后一次prefill所在执行批总共3224 tokens, 耗时却高达4.88秒!

💡 怀疑: 虽然只有700个token, 但是可能chunked prefill执行了两次。

并发瀑布图：9.6万Token长请求独占引擎

⚡ 长请求96564 token，独占5次Prefill

🕒 其他请求全部在等待调度

💡 大请求结束后问题自动恢复



庖丁解牛：Prefill打包机制导致TTFT膨胀

先验知识：引擎每次Prefill最多执行 16384 个token

长请求5 (96564)

Prefill#1

Prefill#2

...#3#4#5

Prefill#6(14644+其他)

Prefill#7

异常请求4 (700+)

等待调度 2.25s

在#6中被打包执行 -> 4.54s

在#7继续 -> 336ms

Prefill#6打包多请求拉满16384 token → 耗时4540ms; Prefill#7仅336ms

异常请求在#6 + #7两次执行，总计 (4540 + 336) = 4876ms

结论：等待2.25s + 被大batch打包执行4.88s = TTFT 7s+

三个案例，三种典型根因

Case 1

性能问题：TPOT破线

现象

Token#125调度耗时6816ms
用户感知6.85s vs TPOT 54ms

根因

非PD分离架构下
大请求Prefill抢占Decode

方案

启用PD分离架构

定位工具

Token分析 + 并发瀑布图

Case 2

精度异常：答非所问

现象

医疗问题回复leetcode答案
日志和网关中看不到异常

根因

首Token为BOS(begin_of_sentence)

方案

修复PD流水线EOS处理逻辑

定位工具

Token分析 + 选词概率分布

Case 3

性能难题：TTFT异常

现象

700 token请求TTFT 7s+
调度2.25s + 执行4.88s

根因

96K token长请求独占5次Prefill
后续打包batch拉满16384 token

方案

长请求隔离

定位工具

Token分析 + 并发瀑布图

💡 共同点：传统监控工具无法发现这些根因，引擎显微镜从Token级精准定位

定位模式统一：统计指标异常 → Token级异常发现 → 并发视角根因定位

从“黑盒猜测”到“透视定位”，平均定位时间从小时级缩短到分钟级

05

社区贡献与展望

● 社区贡献：统一三大引擎可观测Trace标准

10+

提交PR数

合入10+个

2000+

代码变更行

跨三大引擎

3

开源引擎覆盖

vLLM/SGLang/TRT

SGLang

与阿里云共建，贡献Trace可观测能力，对齐vLLM/TRT，形成统一标准

vLLM

贡献Trace可观测能力

**TensorRT
-LLM**

贡献Trace可观测能力，以及可观测指标

2026年将持续同合作伙伴一起将内部优秀经验回馈社区，拓展在AI推理可观测领域的社区影响力



展望

愿景：让每一次推理请求都可追踪、可诊断、可优化

01 Profiling

Attention / MLP / 卡间通信 分层追踪

从Token级下钻到算子级

02 智能诊断

AI自动识别延迟模式并推荐优化方案

基于历史案例库的智能根因匹配

从"人工分析"到"AI辅助诊断"

03 Talk to Engine

动态hook进程方法，暴露类arthas工具

提供合适context

基于AI Agent进行自然语言诊断与性能分析

04 实时告警

基于Token级指标的异常自动检测

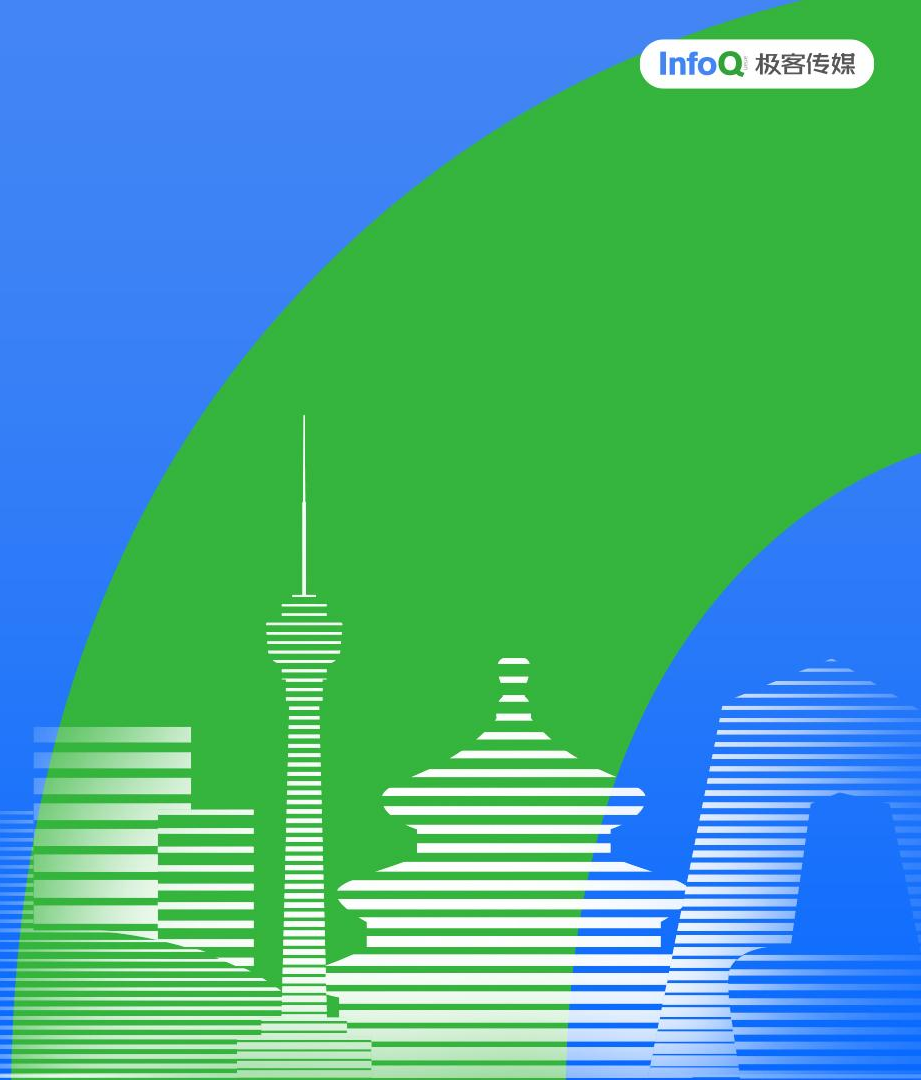
调度延迟 / Prefill抢占 / 重复输出 实时告警

从“事后分析/用户反馈”到“实时感知/主动监控”

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海
站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参
会
咨
询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁洁楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

